A **social interaction** is an exchange between two or more individuals and is a building block of society. Social interaction can be studied between groups of two (dyads), three (triads) or larger social groups

Being social

The word social comes from the Latin socius meaning **"friend."** When you're being social, you're everyone's friend. Go to a social, or mixer, and you might make a lot of new friends.

Conversational rules

Turn-talking - Turn-taking occurs in a conversation when one person listens while the other person speaks. As a conversation progresses, the listener and speaker roles are exchanged back and forth (a circle of discussion)

Back channel - Back channeling is the feedback which a listener gives to a speaker to show that (s)he is following, or understands what the speaker is saying.

Farewell rituals - Rituals and ceremonies help to give meaning to an event. In the case of a farewell ritual, it is the fact of separating from someone.

Implicit & explicit cues - When talking about writing, "**explicit**" means something that is stated plainly, while "**implicit**" refers to something that is implied and not stated directly.

Remote conversations

It's a means of communicating at a distance via electronic tools that let you correspond with people outside of face-to-face communication. It's also referred to as virtual communication, and it's become a quintessential part of the business world.

Tele-presence

Telepresence refers to a set of technologies which allow a person to feel as if they were present, to give the appearance of being present, or to have an effect, via telerobotics, at a place other than their true location.

Co presence

The simultaneous presence of individuals in the same physical/virtual location, not necessarily engaged in face-to-face interaction with each other.
Technologies that enable co-located groups to collaborate more effectively.

EMOTION INTERACTION

The emotional interaction mechanism describes the way emotions affect each other. Emotional interactions are situated intrapersonal, interpersonal, or human-computer interactions necessary for efficient, effective and affective learning.

Emotions and the user experience Expressive & annoying interface Models of emotion Automatic emotion recognition and emotional technologies

Emotional interaction

- What makes us happy, sad, annoyed, anxious, frustrated, motivated, delirious and so on.
    - translating this into different aspects of the user experience.
- Why people become emotionally attached to certain products (e.g. virtual pets).
- Can social robots help reduce loneliness and improve wellbeing?
- How to change human behaviour through the use of emotive feedback.

Emotions & the user experience

- HCI has traditionally been about designing efficient and effective systems.
- Now more about how to design interactive systems that make people respond in certain ways.
- e.g. to be happy, to be trusting, to learn, to be motivated.
- Emotional interaction is concerned with how we feel and react when interacting with technologies.

Emotional model

CLAIM FROM MODEL

- Our emotional state changes how we think.
- When frightened or angry we focus narrowly and body responds by tensing muscles and sweating.
- More likely to be less tolerant.
- When happy we are less focused and the body relaxes.
- More likely to overlook minor problems and be more creative.

Expressive interfaces

- Provide reassuring feedback that can be both informative and fun.
- But can also be intrusive, causing people to get annoyed and even angry.
- Color, icons, sounds, graphical elements and animations are used to make the 'look and feel' of an interface appealing.
- conveys an emotional state.
- In turn this can affect the usability of an interface.
    - people are prepared to put up with certain aspects of an interface (e.g. slow download rate) if the end result is appealing and aesthetic.

Annoying interfaces

MANY CAUSES

– When an application doesn't work properly or crashes.
– When a system doesn't do what the user wants it to do.
– When a user's expectations are not met.
– When a system does not provide sufficient information to enable the user to know what to do.
– When error messages pop up that are vague, obtuse or condemning.
– When the appearance of an interface is garish, noisy, gimmicky or patronizing.
– When a system requires users to carry out too many steps to perform a task, only to discover a mistake was made earlier and they need to start all over again.

GIMMICKS

Amusing to the designer but not the user, e.g.
– Clicking on a link to a website only to discover that it is still 'under construction'

ERROR MESSAGES

Shneiderman's guidelines for error messages include:

- avoid using terms like FATAL, INVALID, BAD
- Audio warnings
- Avoid UPPERCASE and long code numbers
- Messages should be precise rather than vague
- Provide context-sensitive help

WEB ERRORS

COMMON WEB ERROR MESSAGES

- 401 Unauthorized
- 400 Bad Request
- 403 Forbidden
- 404 Not Found
- 408 – Request Time-Out
- 500 Internal Server Error

• 502 Service Temporarily Overloaded
• 503 Service Unavailable

Should computers say they're sorry?
• **Reeves and Naas** (1996) argue that computers should be made to apologize
• Should emulate human etiquette
• Would users be as forgiving of computers saying sorry as people are of each other when saying sorry?
• How sincere would they think the computer was being? For example, after a system crash:
  – "I'm really sorry I crashed. I'll try not to do it again"
• How else should computers communicate with users?

Detecting emotions
EMOTION TECHNOLOGY
• Sensing technologies used to measure GSR, facial expressions, gestures, body movement
• Aim is to predict user's emotions and aspects of their behavior –
• E.g. what is someone most likely to buy online when feeling sad, bored or happy

FACIAL CODING
• Measures a user's emotions as they interact with a computer or tablet.
• Analyses images captured by a webcam of their face.
• Uses this to gauge how engaged the user is when looking at movies, online shopping sites and ads.
• 6 core expressions - sadness, happiness, disgust, fear, surprise and anger.

How to use the emotional data?
If user screws up their face when an ad pops up.
  • feel disgust.
• If start smiling.
  • they are feeling happy.
• Website can adapt its ad, movie storyline or content to match user's emotional state.
• Eye-tracking, finger pulse, speech and words/phrases also analysed when tweeting or posting to Facebook.

Persuasive technology
• Interactive computing systems deliberately designed to change people's attitudes and behaviours (Fogg, 2003).
• A diversity of techniques now used to change what they do or think .
Pop-up ads, warning messages, reminders, prompts, personalized messages, recommendations, Amazon 1-click.
Commonly referred to as nudging.

NINTENDO'S POCKET PIKACHU
Changing bad habits and improving well being.
– Designed to motivate children to be more physically active on a regular basis.
– owner of the digital pet that 'lives' in the device is required to walk, run, or jump.
– If owner does not exercise the virtual pet becomes angry and refuses to play anymore.

TRACKING DEVICES
• Mobile apps designed to help people monitor and change their behavior (e.g. fitness, sleeping, weight)
• Can compare with online leader boards and charts, to show how they have done in relation to their peers and friends
• Also apps that encourage reflection that in turn increase well-being and happiness

Evidence to support anthropomorphism
• **Reeves and Naas** (1996) found that computers that flatter and praise users in education software programs -> positive impact on them.
"Your question makes an important and useful distinction. Great job!".
• Students were more willing to continue with exercises with this kind of feedback.

Virtual characters
Appearing on our screens in the form of:
Sales agents, characters in videogames, learning companions, wizards, pets, newsreaders.
• Provides a persona that is welcoming, has personality and makes user feel involved with them.

Disadvantages
Can lead people into false sense of belief, enticing them to confide personal secrets with chatterbots.
Annoying and frustrating May not be trustworthy.

Implications
• Should we create products that adapt according to people's different emotional states?
  • When people are feeling angry should an interface be more attentive and informative than when they are happy?
• Is Norman right?
  • Designers "can get away with more" for products intended to be used during leisure time than those designed for serious tasks

Summary
• Well-designed interfaces can elicit good feelings in people.
• Aesthetically pleasing interfaces can be a pleasure to use.
• Expressive interfaces can provide reassuring feedback to users.
• Badly designed interfaces make people frustrated, annoyed, or angry.
• Emotional technologies can be designed to persuade people to change their behaviors or attitudes.
• **Anthropomorphism** is the attribution of human qualities to objects.
• Virtual agents and robot pets have been developed to make people feel motivated, reassured, and in a good mood.

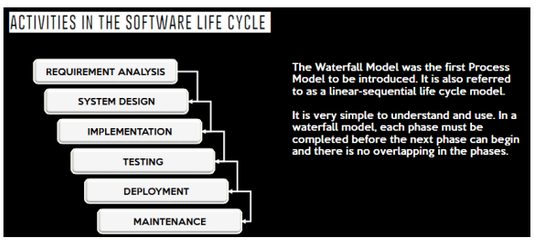
SOFTWARE
Software is a collection of instructions and data that tell the computer how to work. This is in contrast to physical hardware, from which the system is built and actually performs the work.

Software engineering
Software engineering provides a means of understanding the structure of the design process, and that processed can be assessed for its effectiveness in interactive system design. One of the cornerstones of software engineering is the software life cycle.

Software life cycle
Software Development Life Cycle (SDLC) is a process used by the software industry to design, develop and test high quality software.

The SDLC aims to produce a high-quality software that meets or exceeds customer expectations, reaches completion within times and cost estimates.

- SDLC is the acronym of Software Development Life Cycle.
- It is also called as Software Development Process.
- SDLC is a framework defining tasks performed at each step in the software development process.



Requirement Analysis
All possible requirements of the system to be developed are captured in this phase and documented in a requirement specification document.
System Design
The requirement specifications from first phase are studied in this phase and the system design is prepared. This system design helps in specifying hardware and system requirements and helps in defining the overall system architecture.
Implementation
With inputs from the system design, the system is first developed in small programs called units, which are integrated in the next phase. Each unit is developed and tested for its functionality, which is referred to as Unit Testing.
Integration and Testing
All the units developed in the implementation phase are integrated into a system after testing of each unit. Post integration the entire system is tested for any faults and failures.
Deployment of system
Once the functional and non-functional testing is done; the product is deployed in the customer environment or released into the market.
Maintenance
There are some issues which come up in the client environment. To fix those issues, patches are released. Also to enhance the product some better versions are released. Maintenance is done to deliver these changes in the customer environment.

NOTE!
All these phases are **cascaded** to each other in which progress is seen as flowing steadily downwards (like a waterfall) through the phases. The next phase is started only after the defined set of goals are achieved for previous phase and it is signed off, so the name "Waterfall Model". In this model, phases do not overlap.

Waterfall Model -Application
Every software developed is different and requires a suitable SDLC approach to be followed based on the internal and external factors. Some situations where the use of Waterfall model is most appropriate are −
- Requirements are very well documented, clear and fixed.
- Product definition is stable.
- Technology is understood and is not dynamic.
- There are no ambiguous requirements.
- Ample resources with required expertise are available to support the product.
- The project is short.

Advantages
Software Life Cycle: Waterfall model “SDLC - Waterfall Model.” Tutorialspoint, www.tutorialspoint.com/sdlc/sdlc_waterfall_model.htm.
- Simple and easy to understand and use
- Easy to manage due to the rigidity of the model. Each phase has specific deliverables and a review process.
- Phases are processed and completed one at a time.
- Works well for smaller projects where requirements are very well understood.
- Clearly defined stages.
- Well understood milestones.
- Easy to arrange tasks.
- Process and results are well documented.

disadvantages
- No working software is produced until late during the life cycle.
- High amounts of risk and uncertainty.
- Not a good model for complex and object- oriented projects.
- Poor model for long and ongoing projects.
- Not suitable for the projects where requirements are at a moderate to high risk of changing. So, risk and uncertainty is high with this process model.
- It is difficult to measure progress within stages.
- Cannot accommodate changing requirements.
- Adjusting scope during the life cycle can end a project.
- Integration is done as a "big-bang. at the very end, which doesn't allow identifying any technological or business bottleneck or challenges early.

Software Prototyping
Prototype is a working model of software with some limited functionality. The prototype does not always hold the exact logic used in the actual software application and is an extra effort to be considered under effort estimation.

Prototyping is used to allow the users evaluate developer proposals and try them out before implementation. It also helps understand the requirements which are user specific and may not have been considered by the developer during product design.

Throwaway/Rapid Prototyping
Throwaway prototyping is also called as rapid or close ended prototyping. This type of prototyping uses very little efforts with minimum requirement analysis to build a prototype.
Once the actual requirements are understood, the prototype is discarded and the actual system is developed with a much clear understanding of user requirements.

Evolutionary Prototyping
Here the prototype is not discarded and serves as the basis for the next iteration of design. In this case, the actual system is seen as evolving from a very limited initial version to its final release.
Evolutionary prototyping also fits in well with the modifications which must be made to the system that arise during the operation and maintenance activity in the life cycle.

Prototyping -Application
Software Prototyping is most useful in development of systems having high level of user interactions such as online systems. Systems which need users to fill out forms or go through various screens before data is processed can use prototyping very effectively to give the exact look and feel even before the actual software is developed.

Software that involves too much of data processing and most of the functionality is internal with very little user interface does not usually benefit from prototyping. Prototype development could be an extra overhead in such projects and may need lot of extra efforts.
Advantages
Increased user involvement in the product even before its implementation.
• Since a working model of the system is displayed, the users get a better understanding of the system being developed.
• Reduces time and cost as the defects can be detected much earlier.
• Quicker user feedback is available leading to better solutions.
• Missing functionality can be identified easily.
• Confusing or difficult functions can be identified
Disadvantages
• Risk of insufficient requirement analysis owing to too much dependency on the prototype.
• Users may get confused in the prototypes and actual systems.
• Practically, this methodology may increase the complexity of the system as scope of the system may expand beyond original plans.
• Developers may try to reuse the existing prototypes to build the actual system, even when it is not technically feasible.
• The effort invested in building prototypes may be too much if it is not monitored properly. Software Life Cycle : Prototyping

Design rationale
There are many definitions of Design Rationale:
• "Design rationale expresses elements of the reasoning which has been invested behind the design of an artifact" [Shum & Hammond, 1993].
• "Design rationale is the reasoning and argument that leads to the final decision of how the design intent is achieved." "**Design intent** is the `expected' effect or behavior that the designer intended the design object should achieve to fulfil the required function." [Sim & Duffy, 1994]
• "Design rationale means statements of reasoning underlying the design process that explain, derive, and justify design decisions" [Fischer, et. a., 1995]
• Design rationale means "information that explains why an artifact is structured the way that it is and has the behavior that it has" [Conklin, Burgess-Yakemovic, 1995].
• "Design rationales include not only the reasons behind a design decision but also the justification for it, the other alternatives considered, the tradeoffs evaluated, and the argumentation that led to the decision" [Lee, 1997]. Is the information that explains why a computer system is the way it is

Design rationale -Application
• **Design verification**
• — The design rationale can be used to verify if the design decisions and the product itself are the reflection of what the designers and the users actually wanted.
• **Design evaluation**
• — The design rationale is used to evaluate the various design alternatives discussed during the design process.
• **Design maintenance**
• — The design rationale helps to determine the changes that are necessary to modify the design.
• **Design reuse**
• — The design rationale is used to determine how the existing design could be reused for a new requirement with or without any changes in it. If there is a need to modify the design, then the DR also suggests what needs to be modified in the design.
• **Design teaching**
• — The design rationale could be used as a resource to teach people who are unfamiliar with the design and the system.
• **Design communication**
• — The design rationale facilitates better communication among people who are involved in the design process and thus helps to come up with a better design.
• **Design assistance**
• — The design rationale could be used to verify the design decisions made during the design process.
• **Design documentation**
• — The design rationale is used to document the entire design process which involves the meeting room deliberations, alternatives discussed, reasons behind the design decisions and the product overview.

DESIGN RULE
Design rules are the form of standards and guidelines providing direction for designers in order to enhance the interactive properties of the system.
Designing for maximum usability.
– the goal of interaction design.
• **Principles of usability**.
• general understanding.
• **Standards and guidelines**.
• direction for design.
• **Design patterns**.
• capture and reuse design knowledge.

DESIGN RULETYPES OF DESIGN RULE PRINCIPLES TO SUPPORT USABILITY THE GOLDEN RULE
Types of design rules Principles
A **fundamental truth** or proposition that serves as the foundation for a system of belief or behavior or for a chain of reasoning.
• Abstract design rules.
• Low authority.
• High generality.

Standards
An idea or thing used as a measure, norm, or model in comparative evaluations.
• Specific design rules.
• High authority.
• Limited application.
Guidelines
Is a statement by which to determine a course of action.
• Lower authority.
• More general application.

Principles to Support Usability

Dix et al (1998) split the design principles into three main categories.

Learnability
Concerned with interactive system features, which aid novice users to learn quickly and also allows steady progression to expertise.
Flexibility
The variety of ways the user and system exchange information.
Robustness
The level of support provided the user in determining successful achievement and assessment of goal-directed behavior.

Principles to Support Usability
1.Learnability
PREDICTABILITY· SYNTHESIZABILITY · FAMILIARITY · GENERALIZABILITY · CONSISTENCY.

Principles of Learnability
Predictability
This interactive design principle requires a user's knowledge of interaction to be sufficient to determine the outcome of present or future interaction with the system.
For example, closing a document should always allow the user to save changes not saved already.

Synthesisability
Is the ability of the user to assess the effect of past operations on the current state.
For example, when the user makes a payment he expects to receive a receipt, otherwise he will think that the transaction failed or that he is been scammed.

Familiarity
How prior knowledge applies to new system, or make use of the new users past experience with other applications; affordance
For example, A recycle bin is a familiar item in the real world and recycle bin icon immediately suggests its function.

Generalizability
This interactive design principle provides support for users to extend knowledge of specific interaction within, and across applications, to new, but similar situations.
For example, cut/copy/paste/save operations within Microsoft Office applications use of same short-cut keys.

Consistency
To support generalizability, consistency is essential and is probably one of the most widely applied design principle in user interface design. Consistency between application is always favorable, however consistency within an application is essential.

2. Flexibility
DIALOGUE INITIATIVE · MULTI-THREADING · TASK MIGRATABILITY · SUBSTITUTIVITY · CUSTOMISABILITY.

Principles of Flexibility
Dialog initiative
Freedom from system imposed constraints on input dialogue.
For example, user should be able to abandon, suspend or resume. Tasks at any point.

Multithreading
The ability of system to support user interaction for more than one task at a time.
For example, a word processor allow multiple documents to be open, but only one can be worked on at any instant.

Task migratability
The ability to pass control for the execution of a given task so that it becomes either internalized by the user or the system or shared between them.
For example, It is a waste of time for a user to manually check a very long document and correct. A spell checking facility in a word processing application can check words against its own computerized dictionary.

Substitutivity
The extent to which an application allows equivalent input and output values to be substituted for each other.
For example, values in output both digital and analog, output/input.

Customizability
The ability of the user or the system to modify the user interface. Adaptivity can be automated but in order to be able to provide such user-centred system behaviours the system should be trained to distinguish an expert's behaviour from a novice user's behaviour.
The customizability principle supports a user's ability to adjust systems settings or features to a form that best suites the preferred way of usage.

3.Robustness
OBSERVABILITY · RECOVERABILITY · RESPONSIVENESS · TASK CONFORMANCE.

Principles of Robustness
Observability
The extent to which the user can evaluate the internal state of the system from the representation on the user interface.
For example, if the system is performing a time consuming operation, the current status of the operation should be displayed - a web browser will indicate the on-going status of a page download.

Recoverability
The extent to which the user can reach the intended goal after recognizing an error in the previous interaction.
For example, Error recovery can be achieved in two ways, forward (negotiation)
and backward (undo)

Responsiveness
a measure of the rate of communication between the user and the system.

Task conformance
The extent to which the system services support all the tasks the user would wish to perform and in the way the user would wish to perform.

- **Task completeness** - covers whether a system can perform all tasks of interest.
- **Task adequacy** - deals with the user ability to understand the tasks.

For example, When an instantaneous response cannot be given by the system, the system should be able to

indicate to the user that the system has received the request and in processing an appropriate action (see definition of observability).

THE GOLDEN RULE

Schneiderman's 8 Golden Rules of Interface Design

Schneiderman's eight golden rules provide a convenient and succinct summary of the key principles of interface design. They are intended to be used during design but can also be applied, like Nielsen's heuristics, to the evaluation of systems

1. Strive for consistency
2. Enable frequent users to use shortcuts
3. Offer informative feedback
4. Design dialogs to yield closure
5. Offer error prevention and simple error handling
6. Permit easy reversal of actions
7. Support internal locus of control
8. Reduce short-term memory load

THE GOLDEN RULE

Norman's 7 Principles

1. Use both knowledge in the world and knowledge in the head.
2. Simplify the structure of tasks.
3. Make things visible: bridge the gulfs of Execution and Evaluation.
4. Get the mappings right.
5. Exploit the power of constraints, both natural and artificial.
6. Design for error.
7. When all else fails, standardize.